Name : AKASH SAMPATH

Topic : Docker Concepts

Docker File :

```
# 1️⃣ Use official Python image
FROM python:3.10

# 2️⃣ Set working directory inside container
WORKDIR /app

# 3️⃣ Copy requirements file
COPY requirements.txt .

# 4️⃣ Install Python dependencies
RUN pip install -r requirements.txt

# 5️⃣ Copy application files
COPY . .

# 6️⃣ Run the Python application
CMD ["python", "app.py"]
```

Explanation : Line by Line : Docker file :

Line 1 : `FROM python:3.10`

- This is the foundation and this sets the base image and it pulls a pre-configured environment from Docker Hub that has Linux and Python 3.10 already installed
- Starting with a computer that already has Python 3.10 ready to go and you don't have to install python yourself as its already there
- Telling docker start with a computer that already has python 3.10 installed

Line 2 :

`WORKDIR /app`

- Creates and switches to a folder name app inside the container
- Opening a specific folder on your computer where you keep all your project files
- Having a dedicated project folder where keeps everything organized in one place
- Also it is the workspace that sets the working directory where commands like COPY and Run will happen inside this folder
- Opening a specific folder called /app and we will going to do all our work inside this folder from now
- Creates a folder name app inside the container and jumps into it

Line 3 :

`COPY requirements.txt .`

- Copies the requirements.txt file from your computer into the container's /app folder
- This is like getting the list
- This also copies the requirements.txt file from your local computer into the containers current directory
- Copies a specific file from your host machine to the current directory in the image
- You are moving list of libraries like requirement.txt from your computer into the containers folder

Line 4 :

```
RUN pip install -r requirements.txt
```

- Installs all the python packages listed in requirements.txt
- Your python app needs these packages to run properly
- This is like installing tools
- This executes the command to install the python packages listed in your file and builds a layer in the image containing these libraries
- Container looks at your requirements.txt and goes to the internet to download and install those tools

Line 5 :

```
COPY . .
```

- Copies all your application files into the container
- Moving all your project files from your computer into the container folder
- This is like moving your code
- This copies everything from your current local folder into the containers /app folder
- Tools are installed and you are moving your actual python script and any other files into the container so it has something to run
- Copies all files from the local directory into the containers working directory
- Actually moves the rest of your actual code like .py file into the container so it has something to run

Line 6 :

```
CMD ["python", "app.py"]
```

- Tells the container to run python app.py when it starts
- Like double clicking on your python file to run it
- This is a command that actually starts your application
- Known as the launch button
- Provides as the default command for the container and doesn't run during the build phrase and it only runs when you finally start the container
- It tells the container as soon as you start up , run the command python app.py

Docker compose .yml :

```
version: "3.9"

services:

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: akashdb
    ports:
      - "5432:5432"

  app1:
    build: ./app1
```

```
    ports:
      - "5001:5000"
    depends_on:
      - db

  app2:
    build: ./app2
    ports:
      - "5002:5000"
    depends_on:
      - db
```

First Line :

```
version: "3.9"
```

- Specifies which version docker compose syntax to use
- Choosing which edition to use like version 3.9 is a modern stable version with many features
- Also specifies the version of the docker compose file format
- Tells docker which rule to follow like version 3.9 is modern standard that supports most current features

Second Line :

```
services:
```

- Starts the section where we define all our application or containers
- Like a list of all different programs we want to run
- Defines the containers that make up your application and this is the start of all different apps you want to run

Third line :

```
db:
```

- Name this service as database and we are creating a container called db that will handle our data
- This names the first service as db

Fourth Line :

```
image: postgres:15
```

- Uses the official PostgreSQL database version 15 from docker hub
- Downloading and installing PostgreSQL version 15 from an app store
- Pull prebuilt image from docker hub
- Instead of building a database from scratch , docker download a ready to use version of PostgreSQL version 15
- Downloads pre built ready to use Postgres database

Fifth line :

```
environment:
    POSTGRES_USER: myuser
    POSTGRES_PASSWORD: mypassword
    POSTGRES_DB: akashdb
```

- Environment line are the settings or passwords the data base needs to start up
- Sets up database credentials and creates a database
- Tells PostgreSQL create a username with password and make a database called akashdb
- You are setting the username , password and the name of the database folder inside the system

Sixth line :

```
ports:
    - "5432:5432"
```

- Connects ports 5432 on your computer to port 5432 in the container
- Opening a door so programs outside can talk to the database 5432 is PostgreSQL default port number
- Database is opened where your computer can talk to the database on port 5432

Seventh Line :

```
app1:
```

- Names this service app1 and this our first web application

Eight Line :

```
    build: ./app1
```

- Tells docker to build this container using instructions from the app1 folder
- Look in the folder called app1 for dockerfile to build this app
- Instructs docker to look for dockerfile in the app1 folder
- Tells docker to look for a dockerfile in the ./app1 folder to create an image

Nineth Line :

```
  ports:
      - "5001:5000"
```

- Connects port 5001 on your computer to port 5000 in the container and can visit this port using localhost :5001 in your browser
- It opens if you type local host :8001 in your web browser and docker sends that request to the app internal port 5000

Tenth Line :

```
depends_on:
    - db
```

- Makes sure the database starts before this app and don't start this app until the database is ready

Eleventh Line :

```
app2:
  build: ./app2
  ports:
    - "5002:5000"
  depends_on:
    - db
```

- Using app2 folder and port 5002 that needs the database by visiting localhost :5002
- This is second custom application
- Docker builds this app using the files in the /app2 folder
- To reach the application you can go to localhost:5002 or 5000 as it keeps the two apps from bumping into each other
- This app also waits for the database to start before it begins working